TASK 1 — WEB APPLICATION QA REPORT

RealWorld / Conduit • Candidate: Jaswant • 9 November 2026

1. Test overview

Application	react-redux-realworld-example-app (Conduit)
Repository	github.com/gothinkster/react-redux-realworld-example-app
Environment	Windows • Node.js 10.24.1 • npm 6.14.12 • http://localhost:4100
Methods	Manual UI testing, browser Network inspection, source review and Jest test discovery

2. Bug report (five issues, using the supplied template)

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	API error crashes middleware	1. Start app. 2. Load articles while API fails. 3. Observe UI/console.	Expected: graceful error state. Actual: unhandled TypeError reading 'body' of undefined.	High	middleware.js reads error.response.body without checking whether response exists.
2	Articles and tags API return HTTP 530	1. Open app. 2. DevTools → Network → Fetch/XHR. 3. Inspect articles and tags.	Expected: successful API responses. Actual: both requests returned HTTP 530; Cloudflare response headers were observed.	High	Possible upstream/API or edge-service availability issue; exact cause not established.
3	Main stylesheet returns HTTP 404	1. Start app. 2. Inspect Network. 3. Find main.css request.	Expected: stylesheet loads and styles page. Actual: main.css returned HTTP 404 and page appeared unstyled.	Medium	public/index.html references //demo.productionready.io/main.css, unavailable during testing.
4	Login and registration lack required-field validation	1. Open Login or Register. 2. Leave fields empty. 3. Submit.	Expected: block submission and show validation. Actual: inspected components have no required attributes or explicit client-side checks; runtime outcome unverified.	Medium	Login.js and Register.js submit handlers dispatch API requests without explicit required-field validation.
5	Article editor lacks required-field validation	1. Open editor. 2. Leave title, description and body empty. 3. Select Publish Article.	Expected: prevent incomplete submission. Actual: Editor.js submits the article without checking required fields; runtime outcome unverified.	Medium	submitForm constructs and submits the article without validating title, description or body.

3. Root-cause analysis

The application uses custom Redux promise middleware to handle asynchronous API requests. When a request fails, the middleware enters its rejection handler and attempts to read the response body through `error.response.body`. However, a response object is not guaranteed to exist when a request fails before an HTTP response is received. Accessing `body` on an undefined response therefore throws a second TypeError. This interrupts the intended error-handling path and prevents the application from dispatching a consistently structured error action. As a result, the user sees an unhandled rejection rather than a useful error state. The middleware should check that the response and body exist before reading them, and use a safe fallback error payload when they do not. It should then dispatch the failure action consistently so the interface can display a clear message.

4. Test execution and limitations

• The app started locally at http://localhost:4100 using Node.js 10.24.1 and npm 6.14.12.

• Articles and tags requests returned HTTP 530. Therefore signup, login, article creation, editing, deletion and logout could not all be completed end-to-end.

• The browser requested api.realworld.io, while src/agent.js contains API_ROOT = 'https://conduit.productionready.io/api'. A source search found only the latter string in the searched project files. The discrepancy remains unresolved and is not asserted as a confirmed defect.

• Jest checked 40 files and found zero matching test files. This is a test-coverage observation, not one of the five bug rows.

• Source review also found ADD_TAG appends tagInput without trimming, empty-value checks or duplicate checks. A standalone runtime test was not completed because the legacy Babel parser failed on object spread syntax.

• The locally generated package-lock.json remained untracked; no tracked source files were intentionally modified.